

Pico 2 W リアルタイム Opus 音声ストリーミング PoC

RP2350 / Pico 2 W 上で I2S MIC + I2S DAC + LCD + CYW43 Wi-Fi を同時に動かし、Opus 圧縮した音声をクラウドへ送受信する低帯域リアルタイム音声端末 PoC。

教育用・低価格 MCU でも、PIO I2S・DMA・dual-core 分離・Opus 圧縮を組み合わせれば、クラウド接続型のリアルタイム音声端末を構成できる — このファームウェアはそれを実証する。

この PoC が証明すること

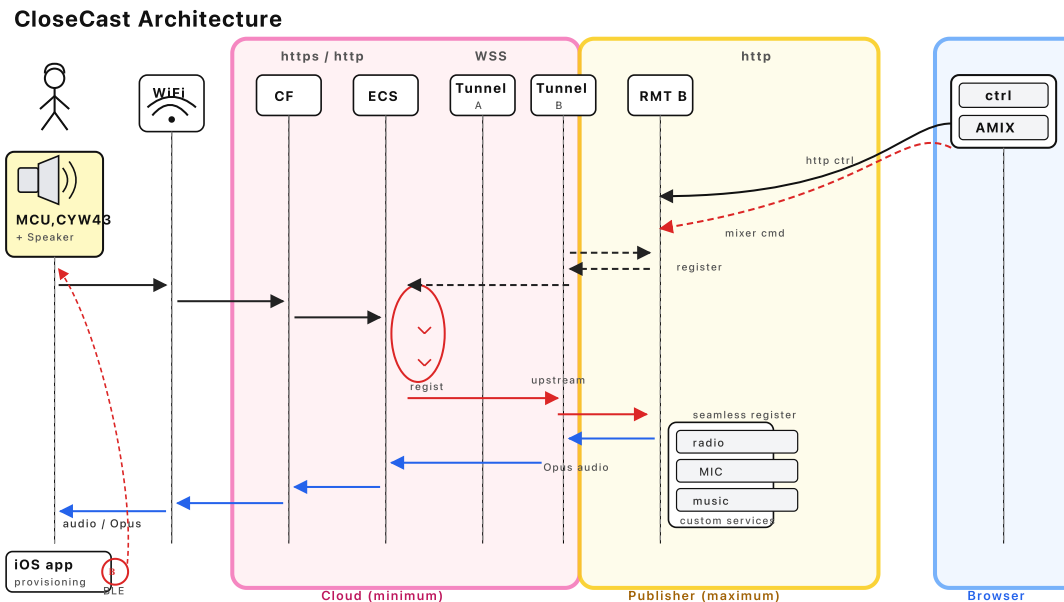
実証項目	詳細
I2S マイク入力	PIO で I2S プロトコルを生成し、MCU 上でリアルタイム音声キャプチャ
I2S DAC/スピーカ出力	PIO + DMA ENDLESS ring で低レイテンシ再生
LCD ステータス UI	ST7789 1.3" SPI LCD で状態表示 + 操作 UI
CYW43 Wi-Fi 常時接続	TLS 常時接続を維持しながら音声処理を並行実行
HTTPS クラウド POST / ポーリング	音声の送信・受信を HTTPS 上で実現
Opus 圧縮ペイロード	PCM 直送ではなく Opus 16-24 kHz mono で帯域を大幅削減
Dual-core 完全分離	Audio/UI と Network を mutex なしで分離、lock-free SPSC ring で接続
BLE プロビジョニング	iOS アプリからゼロタッチで Wi-Fi 設定を書き込み

要するに: RP2350 クラス (\$7) の MCU 1 台で、Opus / TLS / Wi-Fi / I2S Audio が同時に回り、スマホも PC も不要でクラウド音声端末が成立する。

なぜ Pico 2 W で難しいか

- **RAM 520 KB** — TLS バッファ、Opus デコーダ、オーディオ ring、Wi-Fi スタックが全て同居する
- **Wi-Fi と Audio の同時処理** — cyw43_arch_poll と I2S DMA を止めずに回し続ける必要がある
- **I2S を PIO で実装** — RP2350 には I2S ペリフェラルがないため PIO プログラムで生成
- **Opus デコード** — SILK fixed-point デコーダの VLA が ~3 KB のスタックを消費し、デフォルト 4 KB では溢れる
- **バックプレッシャー制御** — クラウド → TCP → ring buffer → DMA を跨ぐフロー制御を自前で構成

システム構成



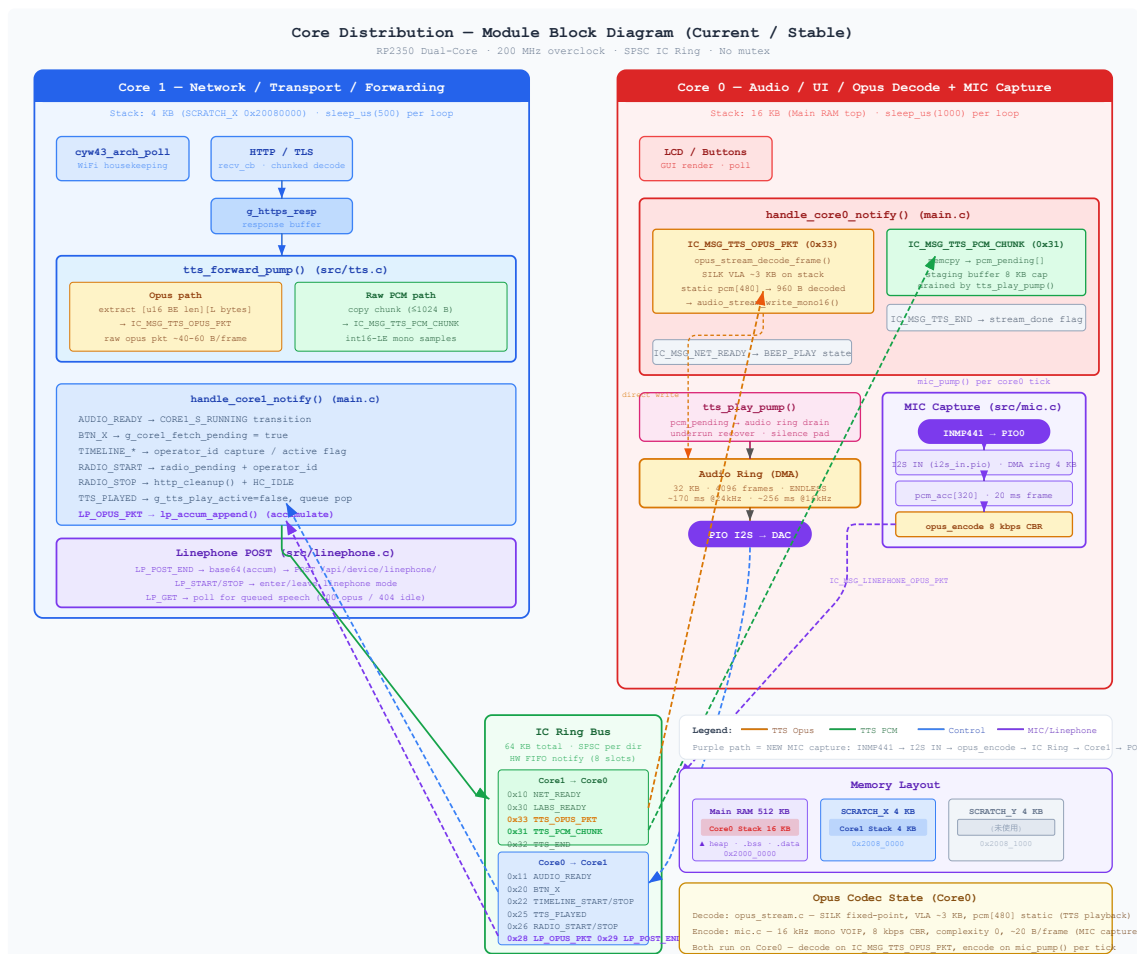
3 ゾーンに分離される。

Cloud (minimum) — デバイス登録・WSS トンネル中継・Opus 音声の下り relay のみ。GPU も CDN キャッシュも持たない。

Publisher (maximum) — radio / MIC / music などの custom service を動的に登録。音声ミキシングとエンコードは Publisher 側で完結し、Cloud はパケットを通過させるだけ。

MCU + Speaker — RP2350 + CYW43 が Wi-Fi 経由で Cloud に TLS 常時接続し、Tunnel 経由で下りてきた Opus パケットを on-chip デコード → I2S DAC で再生する。

Core 分離アーキテクチャ



RP2350 の 2 コアを完全に分離し、mutex なしで音声パイプラインを構成する。

Core 0 (16 KB stack)

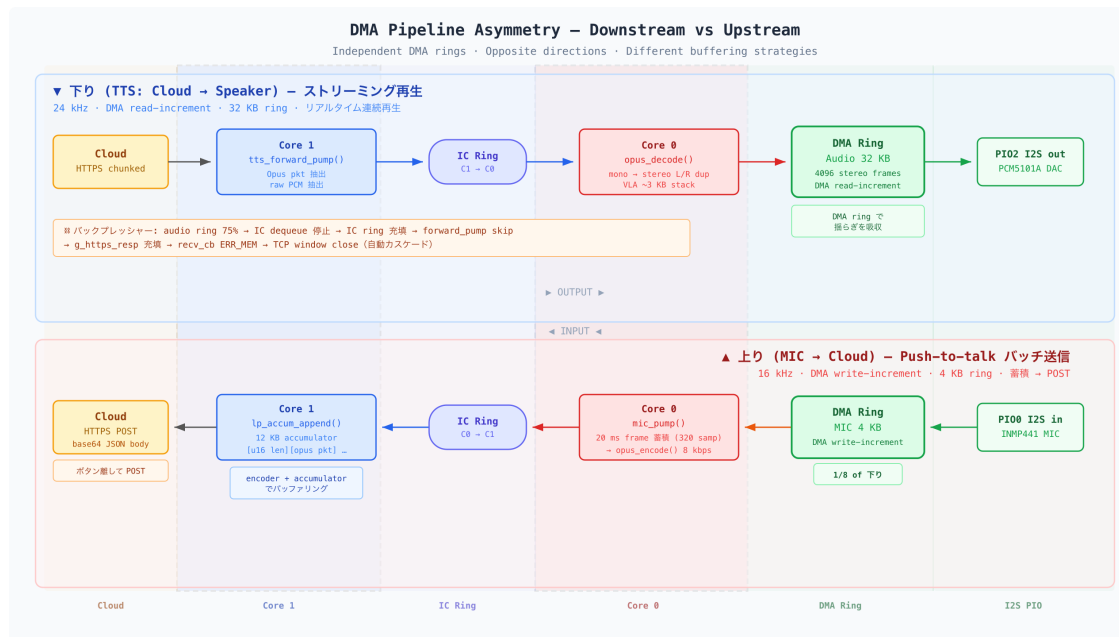
Core 1 (4 KB stack)

	Core 0 (16 KB stack)	Core 1 (4 KB stack)
役割	Audio / UI / Opus Decode	Network / Transport / Forwarding
Stack 配置	Main RAM top	SCRATCH_X
重い処理	opus_decode (SILK VLA ~3 KB)	TLS handshake / cyw43_arch_poll
Loop cadence	1 ms	500 μ s
IC 方向	Consumer (audio data)	Producer (opus pkt / PCM chunk)

Core 間通信は **64 KB の SPSC ring buffer + HW FIFO notify (8 slot)**。バックプレッシャーは audio ring 充填率 75% → IC dequeue 停止 → IC ring 充填 → ic_send_avail==0 → forward_pump skip → g_https_resp 充填 → recv_cb ERR_MEM → TCP window close まで自動カスケードする。

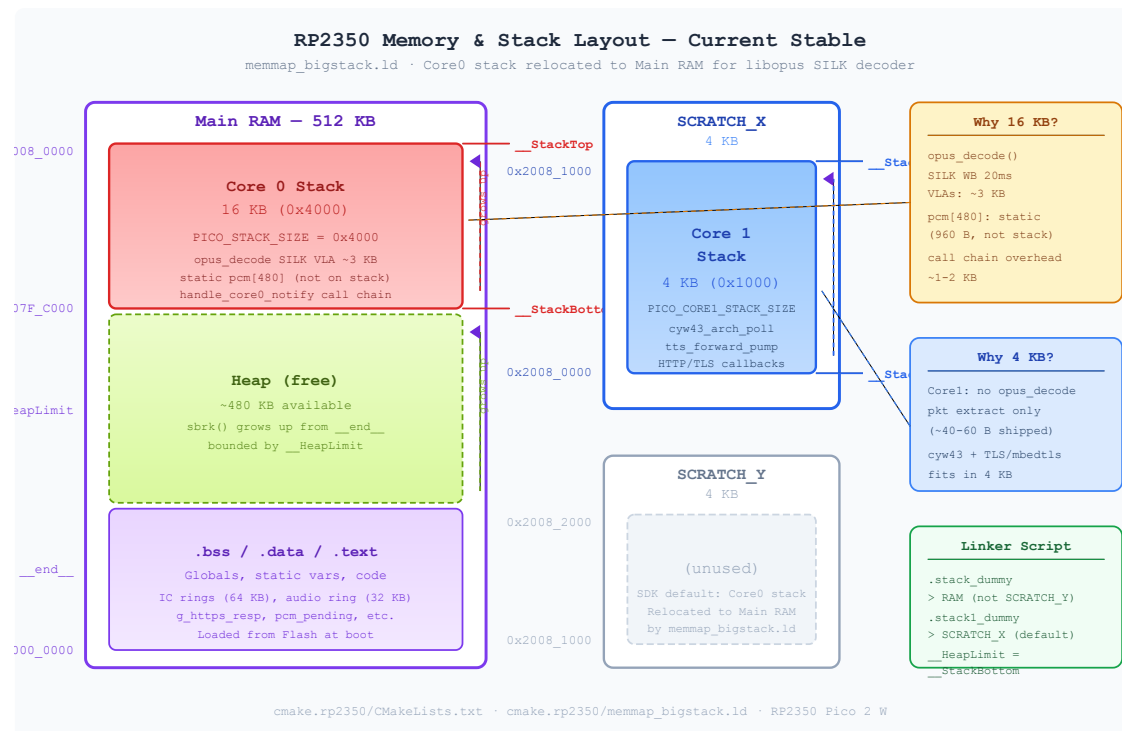
音声パイプライン

上りと下りは **独立した DMA ring** を持ち、**サイズ・方向・バッファリング戦略が非対称** である。



下りはリアルタイム連続再生のため大きな DMA ring (32 KB) で揺らぎを吸収し、上りは発話単位のバッチ送信のため ring は最小限 (4 KB) にして encoder + accumulator 側でバッファリングする。用途が違うので対称にする理由がない。

メモリレイアウト



Core 0 のスタックを SDK デフォルトの SCRATCH_Y (4 KB) から **Main RAM top に 16 KB 確保**して移設。libopus の SILK WB 20 ms fixed-point デコーダが VLA で ~3 KB を消費するため、SCRATCH_Y では溢れる。heap は **__StackBottom** で上限を切る。Core 1 は Opus デコードを行わないため SCRATCH_X 4 KB のまま。

帯域削減効果

PCM 直送と Opus 圧縮の比較:

	PCM 16 kHz 16-bit mono	Opus 16 kbps
ビットレート	256 kbps	16 kbps
削減率	—	93.75%
1分あたり	1.92 MB	120 KB

CYW43 の Wi-Fi スループットと RP2350 の処理余力を考えると、Opus 圧縮は「あれば嬉しい」ではなく、**常時接続を安定させるために必須**の設計判断。

Technical Stack

Layer	What	Why
MCU	RP2350 dual-core 200 MHz	\$7 クラス、Opus リアルタイムデコード可能
Wi-Fi	CYW43 (Pico 2 W)	常時 TLS 接続
TLS	mbedTLS	HTTPS / WSS
Codec	Opus (SILK fixed-point)	低帯域高品質、16–24 kHz mono
Audio Out	PIO I2S → PCM5101A DAC	DMA ring 32 KB, ENDLESS mode
Audio In	PIO I2S MIC	Push-to-talk 音声キャプチャ
IC Bus	SPSC ring + HW FIFO	lock-free, zero-copy
Provisioning	BLE (BTstack)	iOS app でゼロタッチ設定
Display	ST7789 1.3" LCD	状態表示 + 操作 UI

ビルド

前提

- [Pico SDK](#) (≥ 1.3.0)
- ARM GCC toolchain ([arm-none-eabi-gcc](#))
- CMake ≥ 3.12

ビルド手順

```
cd cmake.rp2350
mkdir -p build && cd build
cmake ..
make -j$(nproc)
```

■ 依存ライブラリ

- [libopus](#) — Opus audio decoding (SILK fixed-point)
- [cJSON](#) — JSON parsing